Day 2/5 — Basic Dependency Injection & Bean Wiring

This day is about understanding **how Spring creates objects, manages them as beans, and gives one object the dependencies it needs**.

---

1. What is a Dependency?

A **dependency** is an object/class that another class needs to perform its work.

Simple example

Suppose BookService needs BookRepository:

```
public class BookRepository {

    public void save() {
        System.out.println("Book saved");
    }
}
```

```
public class BookService {

    private BookRepository bookRepository;

    public BookService(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void addBook() {
        bookRepository.save();
    }
}
```

Here:

```
BookService → needs → BookRepository
```

Therefore:

> BookRepository is a dependency of BookService.

Interview point

Q: What is a dependency?

A dependency is an object that a class requires to perform its responsibilities.

---

2. Tight Coupling

Tight coupling happens when a class **creates its own dependency** using new.

```
public class BookService {

    private BookRepository bookRepository = new BookRepository();

    public void addBook() {
        bookRepository.save();
    }
}
```

The problem is that BookService is now directly responsible for creating BookRepository.

```
BookService
    ↓
new BookRepository()
```

Why is this bad?

Suppose later you want:

```
MySQLBookRepository
```

instead of:

```
BookRepository
```

You have to modify BookService.

It becomes difficult to:

- replace implementations
- unit test
- maintain code
- mock dependencies
- follow loose coupling

Interview question

Q: What is tight coupling?

When a class is strongly dependent on a specific implementation and commonly creates that dependency itself, it is tightly coupled.

---

3. Loose Coupling

Instead of creating the dependency inside the class, **receive it from outside**.

```
public class BookService {

    private final BookRepository bookRepository;

    public BookService(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void addBook() {
        bookRepository.save();
    }
}
```

Now:

```
BookService ← BookRepository
             injected from outside
```

BookService doesn't care how BookRepository was created.

This is **loose coupling**.

---

4. What is Dependency Injection?

Dependency Injection (DI) means:

> Instead of a class creating its dependency itself, the dependency is provided/injected into the class from outside.

Without DI

```
public class BookService {

    private BookRepository repository = new BookRepository();
}
```

With DI

```
public class BookService {

    private final BookRepository repository;

    public BookService(BookRepository repository) {
        this.repository = repository;
    }
}
```

The second approach is dependency injection.

---

5. Spring Dependency Injection

Spring takes DI one step further.

Instead of manually doing:

```
BookRepository repository = new BookRepository();

BookService service = new BookService(repository);
```

Spring can create and connect these objects automatically.

```
Spring Container

BookRepository Bean
       ↓
BookService Bean
       ↓
Controller Bean
```

Spring's container is responsible for creating and managing these objects.

---

6. What is a Spring Bean?

A **Spring Bean** is an object that is:

> Created, managed, and controlled by the Spring IoC container.

For example:

```
@Component
public class GreetingService {

    public String greet() {
        return "Hello";
    }
}
```

Spring detects GreetingService and creates an object of it.

Conceptually:

```
GreetingService greetingService = new GreetingService();
```

But **you don't write that new yourself**.

Spring manages it.

---

7. @Component

@Component tells Spring:

> "Create and manage an object of this class as a Spring Bean."

Example:

```
import org.springframework.stereotype.Component;

@Component
public class GreetingService {

    public String greet() {
        return "Hello from GreetingService";
    }
}
```

Now GreetingService becomes a Spring-managed bean.

---

8. @Service

@Service is a specialized form of @Component.

It is normally used for **business/service logic**.

```
import org.springframework.stereotype.Service;

@Service
public class GreetingService {

    public String greet() {
        return "Hello from GreetingService";
```

```
    }
}
```

Spring treats it as a bean.

Conceptually:

```
@Component
   ↑
@Service
```

@Service communicates more clearly:

> "This class contains service/business logic."

When to use which?

Annotation	Typical purpose
@Component	General Spring-managed class
@Service	Business/service logic
@Repository	Database/repository layer
@Controller	MVC controller
@RestController	REST API controller

---

9. Constructor Injection

Constructor injection means providing dependencies through the constructor.

Example:

```
@Service
public class GreetingService {

    public String greet() {
        return "Hello from Service";
```

```
    }
}
```

Controller:

```
@RestController
public class GreetingController {

    private final GreetingService greetingService;

    public GreetingController(GreetingService greetingService) {
        this.greetingService = greetingService;
    }

    @GetMapping("/greet")
    public String greet() {
        return greetingService.greet();
    }
}
```

The flow is:

```
Spring starts
     ↓
Creates GreetingService
     ↓
Creates GreetingController
     ↓
Sees GreetingService is required
     ↓
Injects GreetingService into constructor
```

---

10. Complete Example — GreetingService + GreetingController

Project structure

```
src
└── main
    └── java
        └── com.example.demo
            ├── DemoApplication.java
            ├── GreetingService.java
            └── GreetingController.java
```

GreetingService

```
package com.example.demo;

import org.springframework.stereotype.Service;

@Service
public class GreetingService {

    public String greet() {
        return "Hello from Spring Boot";
    }
}
```

GreetingController

```
package com.example.demo;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class GreetingController {

    private final GreetingService greetingService;

    public GreetingController(GreetingService greetingService) {
        this.greetingService = greetingService;
    }

    @GetMapping("/greet")
    public String greet() {
        return greetingService.greet();
```

```
    }
}
```

Application class

```
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

When you run the application:

```
Spring Boot starts
      ↓
Scans components
      ↓
Finds GreetingService
      ↓
Creates GreetingService bean
      ↓
Finds GreetingController
      ↓
Sees GreetingService dependency
      ↓
Injects GreetingService
      ↓
Application starts
```

Then:

```
GET /greet
```

returns:

```
Hello from Spring Boot
```

---

11. Why final Dependency?

Prefer:

```
private final GreetingService greetingService;
```

instead of:

```
private GreetingService greetingService;
```

Why?

Because after construction, the dependency should normally **not change**.

Constructor:

```
public GreetingController(GreetingService greetingService) {
    this.greetingService = greetingService;
}
```

The final keyword guarantees that the reference is assigned once.

Best practice

```
private final Dependency dependency;
```

with constructor injection.

---

12. Why Constructor Injection Is Preferred

Constructor injection has several advantages.

1. Dependency is explicit

You can immediately see what the class needs:

```
public BookService(BookRepository repository) {
    this.repository = repository;
}
```

2. Supports final

```
private final BookRepository repository;
```

3. Easier unit testing

You can simply create the object yourself:

```
BookRepository repository = new BookRepository();
BookService service = new BookService(repository);
```

Or use a fake/mock:

```
BookRepository mockRepository = mock(BookRepository.class);
BookService service = new BookService(mockRepository);
```

4. Prevents partially initialized objects

A required dependency must be provided when the object is constructed.

---

13. Why Field Injection Is Discouraged

You may see:

```
@Service
public class BookService {

    @Autowired
    private BookRepository repository;
}
```

This is called **field injection**.

It works, but constructor injection is generally preferred.

Problem

You cannot easily create the service with its dependency in a normal unit test:

```
BookService service = new BookService();
```

The repository field hasn't been injected by Spring.

With constructor injection:

```
BookService service = new BookService(repository);
```

The dependency is immediately available.

Interview answer

> Constructor injection is preferred because dependencies are explicit, can be made final, and make classes easier to test without requiring the Spring container.

---

14. Manual new vs Spring Bean

This is a very important concept.

Wrong/mixed approach

```
@Service
public class BookService {

    private final BookRepository repository;

    public BookService() {
        this.repository = new BookRepository();
    }
}
```

If BookRepository is supposed to be Spring-managed, you've bypassed Spring.

You are telling Java:

```
Create this object yourself.
```

instead of:

```
Spring, give me the managed object.
```

Correct

```
@Service
public class BookService {

    private final BookRepository repository;

    public BookService(BookRepository repository) {
        this.repository = repository;
    }
}
```

Now Spring manages the dependency.

---

15. BookService + BookRepository Example

This is an excellent interview example.

Repository

```
@Repository
public class BookRepository {

    public void save(String title) {
        System.out.println("Saving: " + title);
    }
}
```

Service

```
@Service
public class BookService {

    private final BookRepository bookRepository;

    public BookService(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void addBook(String title) {
        bookRepository.save(title);
    }
}
```

Controller

```
@RestController
public class BookController {

    private final BookService bookService;

    public BookController(BookService bookService) {
        this.bookService = bookService;
    }

    @PostMapping("/books")
    public String addBook(@RequestParam String title) {
        bookService.addBook(title);
        return "Book added";
```

```
    }
}
```

Dependency chain:

```
BookController
      ↓
BookService
      ↓
BookRepository
```

Spring creates and wires all three.

---

16. What Happens When @Service Is Removed?

Suppose you have:

```
@Service
public class GreetingService {
}
```

and remove:

```
@Service
```

Now Spring doesn't automatically discover GreetingService as a component.

But your controller still requires:

```
public GreetingController(GreetingService greetingService)
```

Spring tries to create GreetingController.

It asks:

```
Where is GreetingService?
```

No Spring bean exists.

You will typically get an error similar to:

```
Parameter 0 of constructor in GreetingController
required a bean of type 'GreetingService'
that could not be found.
```

The application may fail during startup.

Important

The exact behavior depends on how the class is configured. But in the normal component-scanning setup, removing @Service means Spring no longer automatically registers that class as a bean.

---

17. Spring Startup Logs

When Spring Boot starts, you will see logs similar to:

```
Started DemoApplication in 2.5 seconds
```

At a high level, Spring:

```
1. Starts application
        ↓
2. Creates ApplicationContext
        ↓
3. Scans components
        ↓
4. Finds @Service
        ↓
5. Finds @Repository
        ↓
6. Finds @Controller
        ↓
```

```
7. Creates beans
        ↓
8. Resolves dependencies
        ↓
9. Injects dependencies
        ↓
10. Starts application
```

You don't need to memorize every internal Spring step at this stage.

Understand the concept:

> Spring creates and wires the objects that it manages.

---

18. IoC vs DI

These two terms are closely related but aren't exactly the same.

IoC — Inversion of Control

Normally:

```
Your code → creates objects
```

With Spring:

```
Spring → creates/manages objects
```

Control over object creation is transferred to Spring.

DI — Dependency Injection

DI is a technique used to achieve IoC.

```
Spring
  ↓
creates dependency
  ↓
```

```
injects dependency
  ↓
dependent object
```

Interview answer

> IoC is the broader principle where control of object creation and management is transferred to a framework. Dependency Injection is one way Spring implements IoC.

---

19. Complete Comparison

Tight coupling

```
public class BookService {

    private final BookRepository repository = new BookRepository();

    public void addBook(String title) {
        repository.save(title);
    }
}
```

```
BookService
     |
     └── creates BookRepository
```

Loose coupling

```
public class BookService {

    private final BookRepository repository;

    public BookService(BookRepository repository) {
        this.repository = repository;
    }

    public void addBook(String title) {
```

```
        repository.save(title);
    }
}
```

```
BookService
     ↑
 receives
     |
BookRepository
```

Spring DI

```
@Service
public class BookService {

    private final BookRepository repository;

    public BookService(BookRepository repository) {
        this.repository = repository;
    }
}
```

Spring supplies the dependency.

---

20. Activity 1 — GreetingService Injection

Requirement

Create GreetingService and inject it into GreetingController.

Solution

```
@Service
public class GreetingService {

    public String greet() {
        return "Welcome to Spring Boot";
```

```
    }
}
```

```
@RestController
public class GreetingController {

    private final GreetingService greetingService;

    public GreetingController(GreetingService greetingService) {
        this.greetingService = greetingService;
    }

    @GetMapping("/greeting")
    public String greeting() {
        return greetingService.greet();
    }
}
```

The important part is:

```
private final GreetingService greetingService;
```

and:

```
public GreetingController(GreetingService greetingService) {
    this.greetingService = greetingService;
}
```

That is **constructor injection**.

---

21. Activity 2 — Replace Manual Object Creation

Before

```
@RestController
public class GreetingController {

    private final GreetingService greetingService = new GreetingService();

    @GetMapping("/greeting")
    public String greeting() {
        return greetingService.greet();
    }
}
```

Here the controller manually creates the service.

After

```
@RestController
public class GreetingController {

    private final GreetingService greetingService;

    public GreetingController(GreetingService greetingService) {
        this.greetingService = greetingService;
    }

    @GetMapping("/greeting")
    public String greeting() {
        return greetingService.greet();
    }
}
```

And:

```
@Service
public class GreetingService {

    public String greet() {
        return "Hello";
    }
}
```

Now Spring creates and injects the service.

---

22. Activity 3 — Debug Missing @Service

Start with:

```
@Service
public class GreetingService {

    public String greet() {
        return "Hello";
    }
}
```

Then remove:

```
@Service
```

Your controller still asks for:

```
public GreetingController(GreetingService greetingService)
```

Spring cannot find a GreetingService bean.

Result: **application startup failure** in the normal setup.

This demonstrates that annotations such as @Service are important for component scanning and bean registration.

---

23. Activity 4 — Why Constructor Injection Is Easier to Test

Suppose:

```
public class GreetingService {
```

```
    public String greet() {
        return "Hello";
    }
}
```

Controller:

```
public class GreetingController {

    private final GreetingService greetingService;

    public GreetingController(GreetingService greetingService) {
        this.greetingService = greetingService;
    }
}
```

You can test it without starting Spring:

```
GreetingService service = new GreetingService();
GreetingController controller = new GreetingController(service);
```

That's the key advantage.

The class doesn't require:

```
Spring ApplicationContext
```

just to construct it.

---

24. Common Mistakes

Mistake 1 — Using new for Spring-managed dependencies

```
private final GreetingService service = new GreetingService();
```

Prefer constructor injection.

Mistake 2 — Forgetting @Service

```
public class GreetingService {
}
```

If Spring needs to automatically discover it, make it:

```
@Service
public class GreetingService {
}
```

Mistake 3 — Field injection

```
@Autowired
private GreetingService service;
```

Prefer:

```
private final GreetingService service;

public GreetingController(GreetingService service) {
    this.service = service;
}
```

Mistake 4 — Not using final

Instead of:

```
private GreetingService service;
```

prefer:

```
private final GreetingService service;
```

when the dependency shouldn't change.

Mistake 5 — Confusing class and object

```
GreetingService
```

is a class/type.

```
new GreetingService()
```

creates an object.

Spring manages the **object/bean**, not simply the class definition.

---

25. Interview Questions

Q1. What is Dependency Injection?

Answer:

Dependency Injection is a design technique where an object's required dependencies are provided from outside instead of the object creating them itself.

Q2. Why is DI useful?

Answer:

DI reduces coupling, improves testability, makes dependencies explicit, and makes implementations easier to replace.

Q3. What is tight coupling?

Answer:

Tight coupling occurs when a class strongly depends on a specific implementation, often by directly creating it with new.

Q4. What is loose coupling?

Answer:

Loose coupling means a class depends on abstractions or externally supplied dependencies rather than creating specific implementations itself.

Q5. What is a Spring Bean?

Answer:

A Spring Bean is an object created and managed by the Spring IoC container.

Q6. What does @Component do?

Answer:

@Component tells Spring to detect the class during component scanning and register an instance as a Spring Bean.

Q7. What is @Service?

Answer:

@Service is a specialized @Component used to indicate a service/business-logic class.

Q8. Is @Service fundamentally different from @Component?

Answer:

Not in basic bean registration. Both allow Spring to detect and manage the class. @Service mainly communicates the class's role as a service.

Q9. What is constructor injection?

Answer:

Constructor injection means passing a class's required dependencies through its constructor.

```
public BookService(BookRepository repository) {
    this.repository = repository;
}
```

Q10. Why is constructor injection preferred?

Answer:

Because dependencies are explicit, can be declared final, required dependencies cannot be forgotten during construction, and unit testing becomes easier.

Q11. Why is field injection discouraged?

Answer:

It hides dependencies and makes standalone unit testing harder because dependencies are injected by Spring rather than supplied through the constructor.

Q12. What happens if @Service is removed?

Answer:

Spring normally won't register that class as a bean through component scanning. If another bean requires it, Spring will fail to resolve the dependency and the application will typically fail during startup.

Q13. What is IoC?

Answer:

Inversion of Control means the control of object creation and management is transferred from application code to a framework such as Spring.

Q14. What is the relationship between IoC and DI?

Answer:

IoC is the broader principle, while Dependency Injection is one technique used by Spring to implement IoC.

Q15. Why should dependencies be final?

Answer:

A dependency usually should not change after construction. final ensures the reference is assigned once and communicates that the dependency is required and immutable after initialization.

Q16. Explain BookService needing BookRepository.

Strong interview answer:

> BookRepository is a dependency of BookService because the service needs it to perform repository operations. Instead of creating the repository with new inside BookService, I use constructor injection. Spring creates both beans and injects the BookRepository bean into the BookService constructor. This reduces coupling and makes BookService easier to unit test.

---

26. Interview Scenario

Interviewer:

> Why don't you simply use new BookRepository() inside BookService?

Good answer:

> If I use new, BookService becomes tightly coupled to that particular implementation and controls the creation of its dependency. With constructor injection, the dependency is supplied from outside, so the service is loosely coupled, easier to test, and easier to modify or replace.

---

27. The Most Important Mental Model

Remember this:

Without Spring

```
You create objects
       ↓
```

```
You connect objects
       ↓
You call methods
```

With Spring

```
Spring creates objects
       ↓
Spring connects objects
       ↓
Your classes use their dependencies
```

For example:

```
Spring Container
      │
      ├── GreetingService
      │
      └── GreetingController
                │
                └── GreetingService
```

The controller does **not** say:

```
new GreetingService()
```

Instead:

```
public GreetingController(GreetingService greetingService)
```

and Spring supplies it.

---

28. Day-2 Must Remember

If you're preparing for interviews, remember these **10 points**:

1. **Dependency** = object another class needs.
2. new inside a class often creates **tight coupling**.
3. Passing dependency through constructor promotes **loose coupling**.
4. **DI** = dependencies are provided from outside.
5. **IoC** = Spring takes control of object creation/management.
6. A **Spring Bean** = object managed by Spring.
7. @Component registers a general Spring-managed class.
8. @Service is commonly used for business/service logic.
9. **Constructor injection is preferred**.
10. Prefer **private final + constructor injection** for required dependencies.

One-line interview definition

> **Dependency Injection is a technique where Spring provides a class with the objects it depends on instead of the class creating those objects itself.**